

MODULE 5.1 · Concept 6 of 7 · NEW

Self-Reflection and Iterative Refinement

The Reflexion pattern: generate → evaluate → reflect → retry — beyond basic ReAct

9 min

Duration

B4 – Analyze

Bloom's

L3 – Advanced

Level

TH-000004

Prerequisite

AI-AG-IN-TH-000019

Prof. Sashikumaar Ganesan · IISc Bangalore · AhamX Bodhi

SECTION 1 — WHY

Beyond Basic ReAct



Agents can produce wrong answers and never notice — self-reflection fixes this



Why Basic ReAct Isn't Enough

The agent gives a wrong answer — and has no mechanism to catch it

User: "Write a Python function that returns the first N prime numbers"

Agent (basic ReAct):

Thought: I'll write a function to find primes.

Action: `code_executor(code="def primes(n):\n return list(range(2, n+1))")`

Observation: Function defined.

Answer: "Here's your function: primes(n) returns first N primes"

X WRONG! `range(2, n+1)` returns numbers 2 to n — NOT the first N primes. Agent doesn't check its work.

Basic ReAct has no self-evaluation step. Once the agent generates an answer, it's done. No checking.

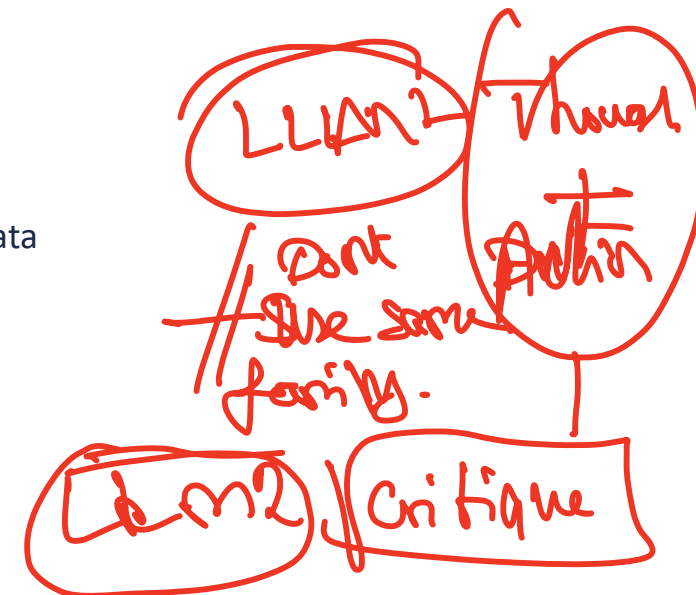
The Reflexion Paper: Shinn et al. (NeurIPS 2023)

Verbal reinforcement learning — agents that critique and improve their own work

Key Insight: Instead of learning from numerical rewards (like RL), agents can learn from verbal feedback — natural language self-critiques stored in memory and used to improve the next attempt.

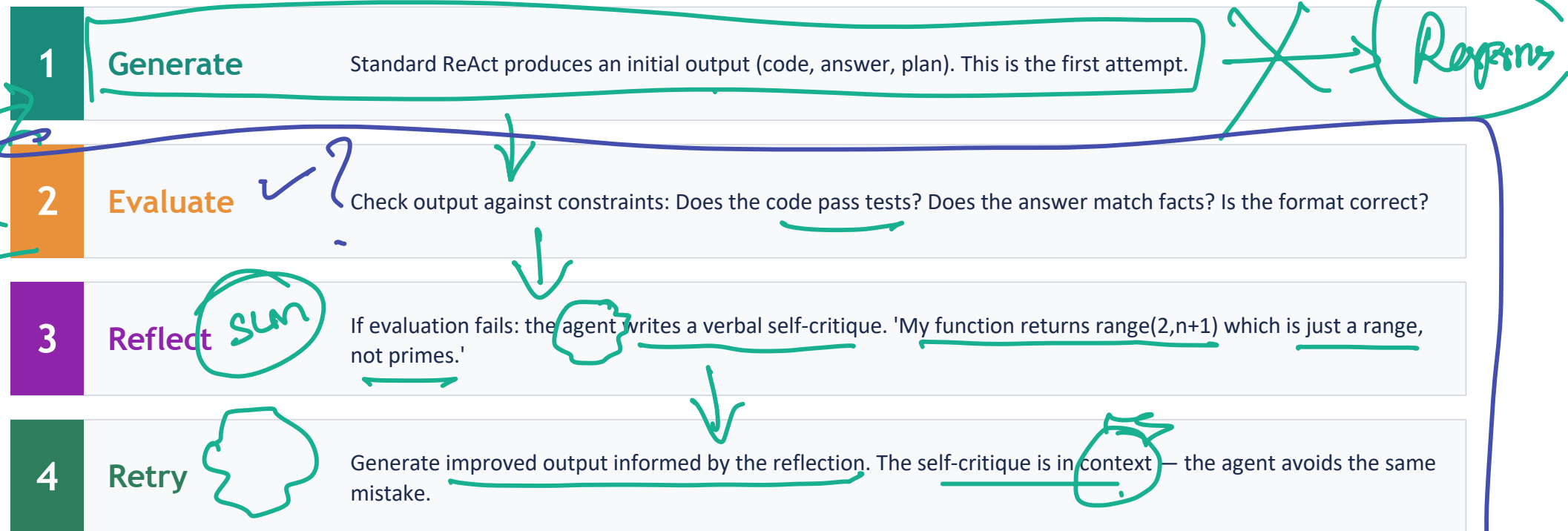
Paper results:

- HumanEval (code generation): 67.0% → 91.0% with self-reflection (+24% absolute)
- ALFWorld (interactive tasks): 75% → 97% with reflection memory
- HotpotQA (multi-hop QA): significant improvement on multi-step reasoning tasks
- Key: improvements come from the agent CRITIQUING its own failures, not from more data



The 4-Phase Loop: Generate → Evaluate → Reflect → Retry

Reflexion adds three steps after the standard ReAct output



SECTION 2 — HOW

The Four Phases in Detail

Each phase has specific implementation strategies



Generate Phase: Standard ReAct Output

The first attempt — often good enough, sometimes wrong

The Generate phase is standard ReAct — nothing new here:

Thought: I need to write a function that returns the first N primes.

Action: `code_executor(code="""def first_n_primes(n):`

`primes = []`

`candidate = 2`

`while len(primes) < n:`

`if all(candidate % p != 0 for p in primes):`

`primes.append(candidate)`

`candidate += 1`

`return primes`

`print(first_n_primes(10))""")`

Observation: `[2, 3, 5, 7, 11, 13, 17, 19, 23, 29]`

This looks correct — but we should VERIFY before delivering to the user. That's the Evaluate phase.

Evaluate Phase: Check Output Against Constraints

Does the output actually satisfy the requirements?

Three evaluation strategies:

①

Constraint Validation

Check against hard rules: format correct? length within limit? schema valid?
All test cases pass?

Run primes(10) — does it return exactly 10 primes? Are all results actually prime?

②

LLM-as-Judge

Second LLM call evaluates quality: 'Rate this output 1–5 for correctness, completeness, clarity.'

'Does this function correctly compute primes? Score: 5/5. Edge cases: primes(0)=[]? primes(1)=[2]?'

③

Ground Truth Comparison

Compare output against known correct answer or test cases provided by the user.

User provided test: first_n_primes(5) should return [2,3,5,7,11]. Does it match?

RAG

Reflect Phase: Agent Critiques Its Own Work

Verbal self-feedback that guides the next attempt

Self-Reflection (when evaluation FAILS):

Reflection: "My function `range(2, n+1)` simply returns consecutive integers starting from 2, not prime numbers. I confused 'first N numbers starting at 2' with 'first N primes'. I need a proper primality check – either trial division or a sieve. The corrected approach should:

1. Start with candidate = 2
2. Check if candidate is prime (divisible only by 1 and itself)
3. If prime, add to result list
4. Stop when list has N items

I should also handle edge cases: `n=0` should return `[]`, `n=1` should return `[2]`."

This reflection stays in the context window for the Retry phase. The agent now 'knows' what went wrong.

Retry Phase: Improved Output Informed by Reflection

The self-critique prevents repeating the same mistake

With the reflection in context, the agent generates a corrected version:

```
Retry: code_executor(code="""def first_n_primes(n):
    if n <= 0: return []
    primes = []
    candidate = 2
    while len(primes) < n:
        is_prime = all(candidate % p != 0 for p in primes if p*p <= candidate)
        if is_prime: primes.append(candidate)
        candidate += 1
    return primes
assert first_n_primes(0) == []
assert first_n_primes(5) == [2,3,5,7,11]
print("All tests passed!")""")
```

The corrected function handles edge cases, uses proper primality testing, and includes assertions.

SECTION 3 — STRATEGIES AND TRADEOFFS

Self-Correction Strategies and When NOT to Reflect

Making reflection practical in production agents

Strategy 1: Constraint Validation

Cheapest form of evaluation — hard rules that can be checked programmatically

Constraint Type	How to Check	Cost	Example
Format/Schema	JSON schema validation	\$0 (code)	Response must be valid JSON with 'answer' field
Length	Character/word count	\$0 (code)	Summary must be < 200 words
Test cases	Run code, check output	\$0 (code)	primes(5) must return [2,3,5,7,11]
Factual consistency	Compare against retrieved data	\$0 (code)	Answer references only data in Observations
Safety	Blocklist/classifier check	\$0 (code)	No PII, no harmful content in response

Always try constraint validation FIRST. It's free and catches the most common errors.

Strategy 2: LLM-as-Judge + Strategy 3: Retry Budgets

When constraints aren't enough — and when to stop retrying

LLM-as-Judge Self-Evaluation

most Popular.

A second LLM call evaluates the output:

'Rate this response for:

- Correctness (1-5)
- Completeness (1-5)
- Clarity (1-5)

If any score < 3, explain what's wrong.'

Cost: ~\$0.001 per evaluation call

Use when: constraints can't capture quality

Retry Budgets: Prevent Infinite Loops

CRITICAL: Set a maximum retry count!

```
max_retries = 3 # Hard limit
```

After 3 failed attempts:

- Return best attempt with caveat
- Escalate to human
- Log failure for analysis

Without a budget, reflection loops forever.

This is the #1 deployment risk.

Reflexion vs. Basic ReAct: Comparison

When is the extra cost of self-reflection worth it?

Dimension	Basic ReAct	ReAct + Reflexion
Accuracy	First-attempt accuracy only	Improves with each retry (+24% on code)
Cost	Lower (no evaluation step)	~2–4× per task (evaluation + retries)
Latency	Faster (fewer iterations)	Slower (evaluate + reflect + retry per attempt)
Reliability	Inconsistent on hard tasks	Much more consistent on complex tasks
Debuggability	Read trace once	Read reflection to understand self-corrections
Best for	Simple lookups, single-step	Code generation, writing, complex reasoning

Use Reflexion for high-stakes tasks where correctness matters more than speed or cost.

Key Takeaways

The essential points from this concept

- Basic ReAct doesn't self-check.** Reflexion adds: generate → evaluate → reflect → retry.
- Shinn et al. (2023) showed +24% on code generation** with verbal self-reflection.
- Three evaluation strategies:** constraint validation (\$0), LLM-as-judge (~\$0.001), ground truth comparison.
- Always set retry budgets (max 3).** Without them, reflection loops forever — the #1 deployment risk.
- Use Reflexion for high-stakes tasks** where correctness matters more than speed.

Next: AI-AG-IN-CD-000001 — Implementing ReAct Agents (hands-on coding lab)

AI-AG-IN-TH-000019 Complete

Self-Reflection and Iterative Refinement

Concept 6 of 8

Advanced Certification in Agentic & Generative AI · IISc Bangalore

Prof. Sashikumaar Ganesan · AhamX Bodhi